Phone Controlled Car

Dobrete Andrei-Robert, Mihalache Denisa Ana-Maria

University: Polytechnic University of Bucharest; Faculty: Industrial Engineering and Robotics; Specialization: Robotics; Year of study: 1; e-mail: andy3153@protonmail.com, mihalachedenisa8@gmail.com

Scientific leader: prof. Ivan Andrei Mario

Project link: https://github.com/Andy3153/phone-controlled-car

Abstract

Summary: This project implements a Bluetooth-enabled vehicle controlled via smartphone, providing a flexible foundation that can be adapted for numerous use cases.

Key words: car, robot, bluetooth, remote control.

Contents

Contents 1

1 Introduction 1

2 Materials and Methods 2

2.1 Hardware Components . . . 2

2.2 Circuit Design . . . 2

2.3 Physical Implementation . . . 3

2.4 Software and Tools . . . 3

2.4.1 IDE . . . 3

2.4.2 Libraries used by the code . . . 3

2.4.3 Mobile application used for control . . . 3

2.4.3.1 Configuring the mobile application . . . 3

2.5 Source code . . . 4

1 Introduction

The rapid development of low-cost microcontroller platforms has created new possibilities for the development of versatile robotic systems. The Arduino ecosystem has emerged as the most popular open-source prototyping platform because of its accessible hardware architecture, extensive library support and the large community around it. By integrating Bluetooth communication, it becomes possible to create a wireless link between a microcontroller-based vehicle and a smartphone, enabling real-time control via a dedicated mobile application.

This paper presents the design, implementation and source code of a smartphone controlled robotic car built upon the Arduino platform. The system uses the `Serial Port Profile (SPP)` of the Bluetooth standard to transmit directional control signals from a mobile device to the Bluetooth module (`HC-05`), which gets parsed by the microcontroller to drive a motor driver module (`L298N`) that actuates the vehicle's DC motors. The modularity, simplicity and extensibility of the architecture allow it to serve as a foundational platform for a wide range of applications. These include, but are not limited to:

- remote-controlled (RC) navigation,
- autonomous maze-solving through wall-following algorithms,
- obstacle avoidance via ultrasonic or infrared proximity sensors,
- line-following using reflectance sensor arrays,
- robotic manipulation through the integration of a robotic arm.

The flexibility of the platform makes it well-suited for both educational environments and practical deployment scenarios.

2 Materials and Methods

2.1 Hardware Components

The main required hardware components of the car are listed as follows:

component	model	link
battery cells	18650	optimusdigital.ro/ro/acumulatori-li-ion/3152-acumulator-samsung-li-ion-3000-mah-18650-inr18650-30q.html
bluetooth module	HC-05	optimusdigital.ro/ro/wireless-bluetooth/153-modul-bluetooth-master-slave-hc-05-cu-adaptor.html
development board	Arduino Uno	optimusdigital.ro/ro/placi-avr/4561-placa-de-dezvoltare-compatibila-cu-arduino-uno-r3-atmega328p-atmega16u2-cablu-50-cm.html
motor driver module	L298N	optimusdigital.ro/ro/drivere-de-motoare-cu-perii/145-driver-de-motoare-dual-l298n.html
motor with reduction gears and wheels	-	optimusdigital.ro/ro/motoare-altele/139-motor-cu-reductor-si-roata.html

Table 2.1: Hardware components of the car

2.2 Circuit Design

The circuit schematic, designed and rendered using the Fritzing electronic design automation (EDA) tool, is presented below:

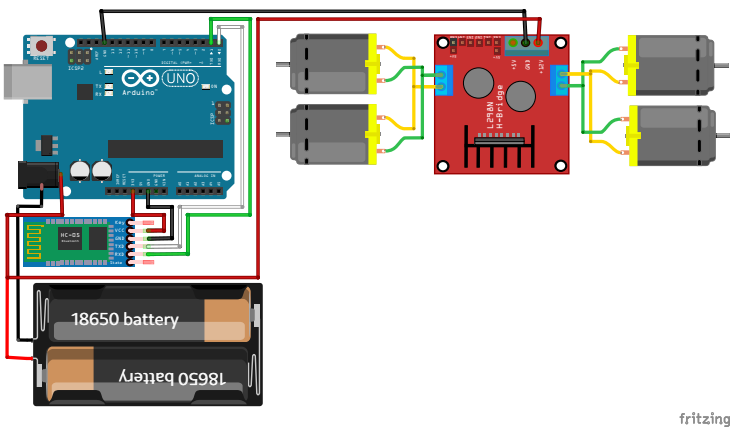


Figure 2.1: Circuit schematic

2.3 Physical Implementation

The fully assembled prototype of the car is shown below, showcasing the physical integration of all hardware components.

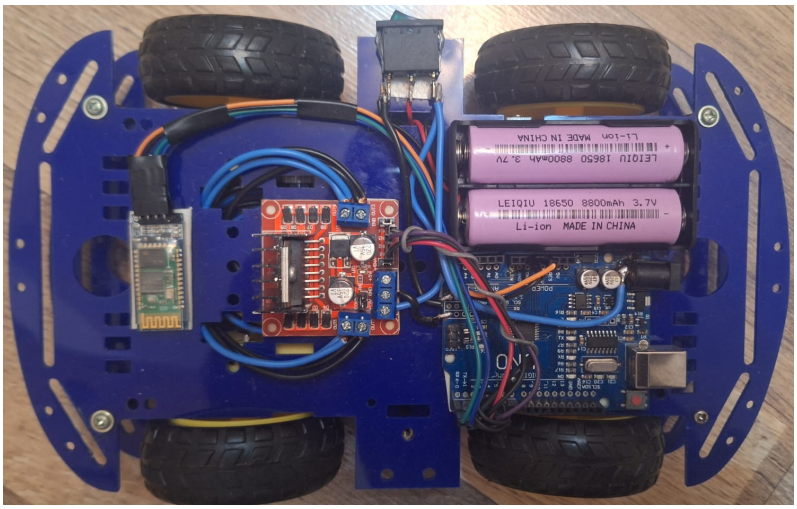


Figure 2.2: Car prototype

2.4 Software and Tools

2.4.1 IDE

Software development was carried out using the Arduino build toolchain. While the Arduino IDE (`docs.arduino.cc/software/ide`) provides an accessible interface for compilation and flashing, the project used Arduino-Makefile by Sudar Muthu (`github.com/sudar/Arduino-Makefile`), allowing development in a custom environment (in this case, Neovim).

2.4.2 Libraries used by the code

To interface with the dual H-bridge motor driver, an external library was used: the open-source `L298N` library by Andrea Lombardo (`github.com/AndreaLombardo/L298N`), specifically utilizing the `L298NX2.h` header, which provides a high-level abstraction for the simultaneous control of two independent motor channels.

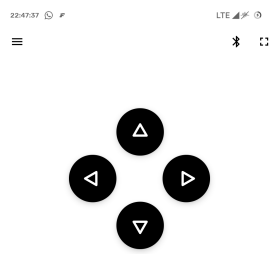
2.4.3 Mobile application used for control

A variety of mobile applications are compatible with the system. For the purposes of this study, the primary application used for testing and validation was the first listed below. The applications that were tested are listed as follows:

2.4.3.1 Configuring the mobile application

The mobile application requires the configuration of four directional control buttons, each mapped to a distinct single-character command transmitted over the Bluetooth serial link: `N`, `S`, `W`, and `E` corresponding to forward, reverse, left, and right motion respectively. An additional button is assigned the null command character `0`, which triggers a full motor halt.

An example configuration is shown below:



app name	developer	link
Arduino Bluetooth Controlled Joystick	Uncia Robotics	`play.google.com/store/apps/details?id=uncia.robotics.joystick`
Arduino bluetooth controller	Giumig development apps	`play.google.com/store/apps/details?id=com.giumig.apps.bluetoothserialmonitor`
Bluetooino: Arduino Bluetooth	Strike Software	`play.google.com/store/apps/details?id=strikesoftware.bluetooino`
Arduino Bluetooth	Kharis Digital	`play.google.com/store/apps/details?id=com.arbl.arduinobluetooth`

Table 2.2: Applications tested with the project

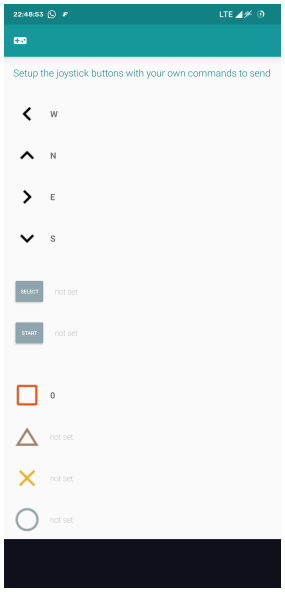


Figure 2.4: Button mappings

2.5 Source code

Given the use of an Arduino-compatible development board, the source code was written in the Arduino programming language, a C/C++-based dialect built on top of the AVR-GCC toolchain, augmented by the Arduino core libraries for hardware abstraction.

The complete source code is publicly available at the repository linked in the project title (`https://github.com/Andy3153/phone-controlled-car`), at path `src/main.ino`. For completeness, the full source code is also shown below:

```
// vim: set fenc=utf-8 ts=4 sw=0 sts=0 sr et si tw=0 fdm=marker fmr={{{,}}}:

// {{{ Libraries
#include <L298NX2.h> // motor driver module
// }}}

// {{{ Variables
// {{{ Declare pins
const int IN1 = 2,
          IN2 = 3,
          IN3 = 4,
          IN4 = 5;
// }}}

// {{{ Declare motors on the L298N
L298NX2 motors(IN1, IN2, IN3, IN4);
```

```
// }}}

// {{{ Bluetooth state
int state;
// }}}
// }}}

// {{{ Setup
void setup()
{
    // {{{ Set pin modes
    // For the L298N inputs
    pinMode(IN1, OUTPUT);
    pinMode(IN2, OUTPUT);
    pinMode(IN3, OUTPUT);
    pinMode(IN4, OUTPUT);
    // }}}

    // {{{ Initialize serial communication
    Serial.begin(9600);
    // }}}
}
// }}}

// {{{ Loop
void loop()
{
    if(Serial.available() > 0) state = Serial.read(); // save the state into a varable

    switch(state)
    {
        // {{{ Move forward
        case 'N':
            motors.forward();
            break;
        // }}}

        // {{{ Move backward
        case 'S':
            motors.backward();
            break;
        // }}}

        // {{{ Rotate right
        case 'E':
            motors.forwardA();
            motors.backwardB();
            break;
        // }}}

        // {{{ Rotate left
        case 'W':
            motors.backwardA();
            motors.forwardB();
            break;
        // }}}

        // {{{ Stop motors
        case '0':
            motors.stop();
            break;
        // }}}
    }
}
// }}}
```